

3C7 Digital Systems Design - Assignment 2

Harry Leddy – 23372611

Contents

1.0: Introduction:	2
2.0: System Architecture:	2
2.1: LFSR Architecture:	2
2.2: FSM Architecture:	2
2.3: Counter Architecture:	2
2.4: Overall Signal Flow:	3
3.0: LFSR Design:	4
3.1: LFSR Specifications:	4
3.2: LFSR Testbench:	5
4.0: FSM Design:	6
4.1: Initial Implementation:	6
4.2: Corrected Implementation:	7
4.3: State Minimisation:	7
4.4: FSM Testing:	8
5.0: Counter Design:	9
5.1: Counter Testbench:	9
6.0: Implementation:	10
6.1: Module Implementation & Hierarchy:	10
6.2: Design Decisions:	11
6.3: Implementation Testing:	11
6.4: Clock Scaling:	13
7.0: Reporting & Testing:	14
7.1: Utilisation Report:	14
7.2: Timing Report:	14
8.0: Demo:	15

1.0: Introduction:

This assignment involves implementing a system to detect the number of occurrences of a binary codeword in a pseudo-random bitstream. The bitstream will be generated using an 18-bit Linear Feedback Shift Register (LFSR) with one bit being processed each clock cycle. A Finite State Machine (FSM) will be used to identify each instance of the target codeword in the bitstream.

The specific codeword that was assigned for this design is 10010, a 5-bit sequence. This system is required to detect all occurrences of this sequence in the full cycle of the LFSR, including overlapping instances.

The overview of the architecture consists of three main components:

- **18-bit LFSR:** Generates a pseudo-random bitstream with a sequence length of $2^{18} - 1$.
- **FSM:** A Moore-type finite state machine that detects the codeword by tracking partial matches within the bitstream.
- **Binary Counter:** Increments on each detection event, recording the total number of occurrences.

The system architecture is further described in Section 2 of this report.

2.0: System Architecture:

2.1: LFSR Architecture:

The LFSR module generates the pseudo-random input used by the system. An 18-bit LFSR was implemented, producing a deterministic sequence of length $2^{18} - 1 \approx 262,143$. The least significant bit (LSB) of the LFSR output is used as the input to the FSM, providing one bit per clock cycle. This allows the sequence detector to process the input as a bitstream.

2.2: FSM Architecture:

The sequence detector is implemented as a Moore-style finite state machine (FSM). It receives the bitstream from the LFSR and tracks partial matches to the target codeword 10010. When the full sequence is detected, the FSM outputs a HIGH signal for one clock cycle. The FSM is designed to allow overlapping detections, which allows consecutive occurrences to be counted correctly.

2.3: Counter Architecture:

The counter module records the total number of detected codewords. It increments by one each time the FSM output signal is detected to be HIGH. The counter is synchronous with the system clock and is reset at the start of operation. Its output represents the total number of codeword occurrences and is used by the seven-segment display for visualisation.

2.4: Overall Signal Flow:

Figure 1 outlines the overall signal flow of the system. The LFSR generates the 18-bit pseudo-random sequence, with its least significant bit used as the input to the FSM. The FSM processes this bitstream and sets the detection signal to HIGH whenever the target codeword is identified.

This signal is used to increment the counter, which accumulates the total number of detections over one full LFSR cycle. The counter output is then routed to the seven-segment display for visualisation. The full LFSR output is also routed to the Basys-3 board's LEDs for observation purposes. All modules operate under a common clock and reset signal, as shown in the diagram.

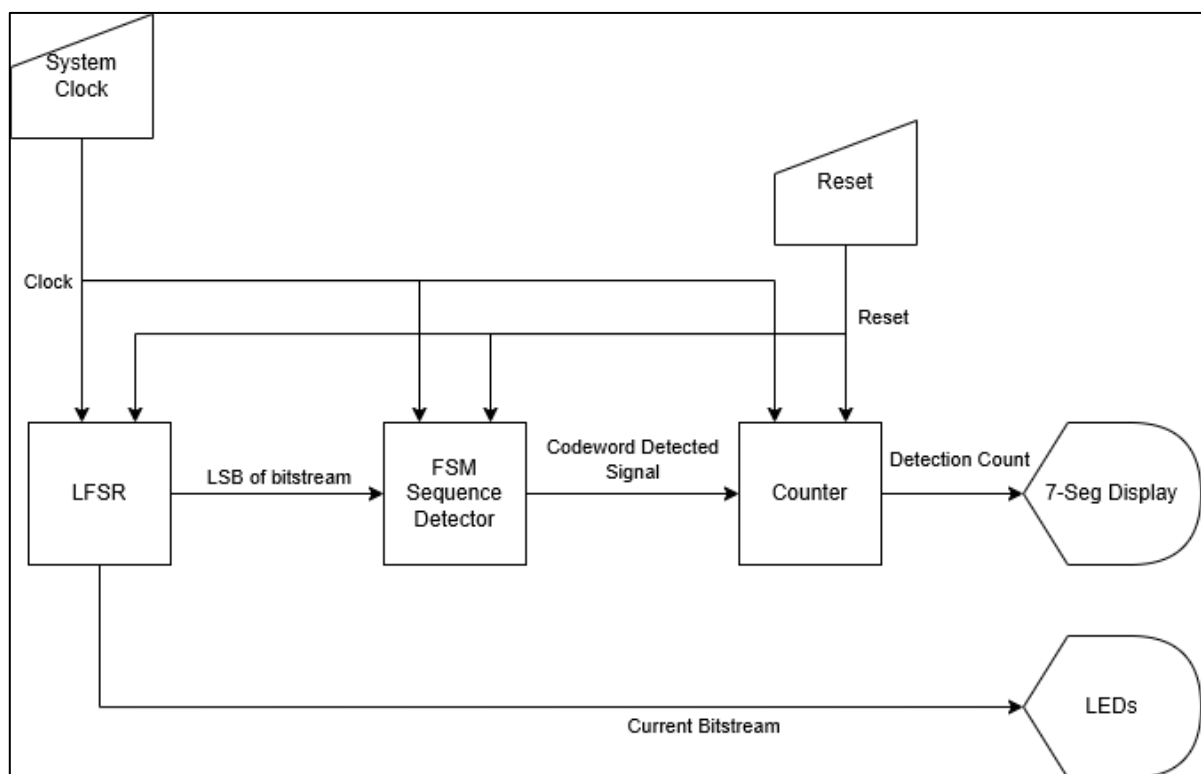


Figure 1 - System Architecture Block Diagram

3.0: LFSR Design:

3.1: LFSR Specifications:

The LFSR designed and tested as part of “Lab F” was reused. The primary changes that had to be made to the LFSR design were as follows:

- Changing the tap-length from 23 to 18 bits.
- Adjusting the tap-positions for the feedback comparison.
- Modifying the seed value.

The tap-positions for calculating the feedback bit were referenced from the supplied document “*Efficient Shift Registers, LFSR, Counters, and Long Pseudo-Random Sequence Generators*” from XILINX. For the 18-bit LFSR the recommended tap positions were 18 and 11. Therefore, using zero-based indexing, the feedback bit was calculated as follows:

$$Q_{fb} = Q[17] \oplus Q[10]$$

These tap positions ensure a maximal-length LFSR, producing a $2^{18} - 1$ length sequence before repeating.

As instructed in the assignment brief, to calculate the starting seed both the board number and last two digits of the student number had to be converted to hexadecimal representation:

Final Two Digits of Student Number:

$$11_{10} = B_{16} = 0000\ 1011$$

Board Number:

$$18_{10} = 12_{16} = 0001\ 0010$$

These were then converted to binary to be bitwise XNOR'd in order to calculate the seed value, this can be seen in Figure 2.

Therefore the new seed value can be given as:

$$\begin{aligned} \text{Seed Value} &= E6_{16} \\ &= 00\ 0000\ 0000\ 1110\ 0110 \end{aligned}$$

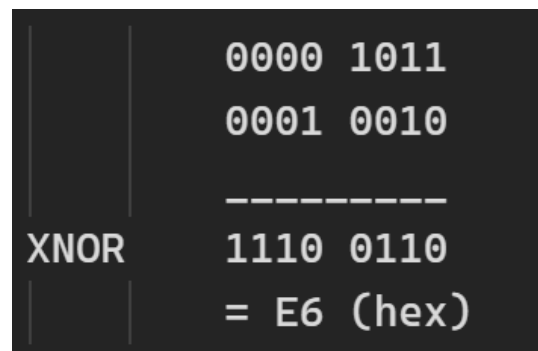


Figure 2 - Seed Value Calculation

This LFSR design can be used to generate $2^N - 1$ different bit-combinations:

$$2^{18} - 1 = 262,143 \text{ combinations}$$

If operating at 1 Hz as in Lab F, a single cycle of all LFSR values would take approximately:

$$4,369 \text{ minutes} \approx 72 \text{ hours} \approx 3 \text{ days}$$

This is far too long for practical real-world testing. Thus, validation of this design was completed by using simulations instead.

3.2: LFSR Testbench:

The testbench had a clock cycle of 2 ns, with a 10-cycle reset period during initialisation, this would take 20 ns to complete.

It was therefore calculated that the testbench would have to be simulated for:

$$t_{\text{testbench}} = 20 + 2(262,143) = 524,306 \text{ ns}$$

When simulated, the maximum-tick-reset signal went to high after 524,306 ns, as expected. This indicates that the LFSR completed a full cycle and would then begin the deterministic sequence of values again. This signal can be seen to trigger in Figure 3.

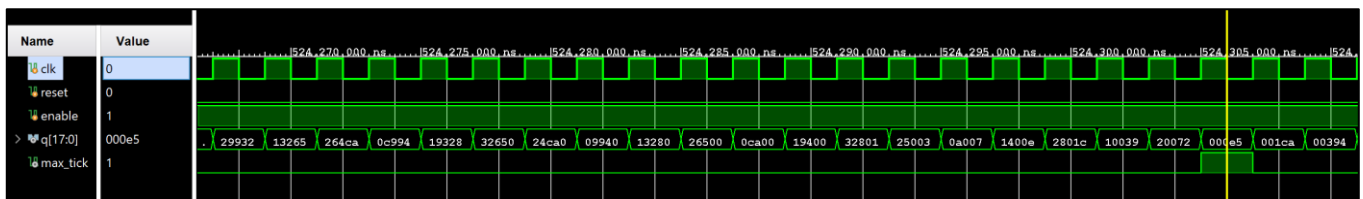


Figure 3 - Max-Tick-Reset LFSR

The general function of the LFSR was also re-tested. The expected behaviour was confirmed, with existing bits shifting left at each clock cycle and the new incoming bit being set by the feedback bit. This behaviour only begins after the initialising 10 clock cycles of the testbench.

This behaviour can be seen in Figure 4.

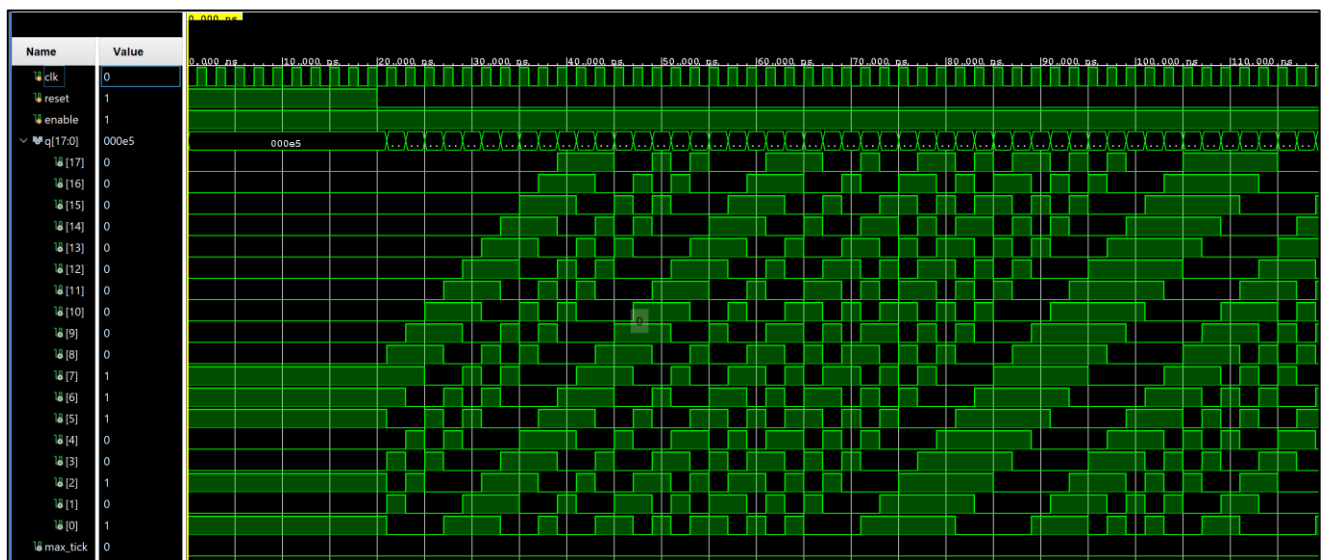


Figure 4 - 18-bit LFSR Testbench

4.0: FSM Design:

The sequence detector was implemented as a **Moore finite state machine**. The target codeword for this assignment was 10010. As it's a Moore machine, the output depended only on the current state, rather than directly on the input bit.

The FSM was designed such that each state represents the longest prefix of the codeword matched so far. The state definitions are as follows:

- **State A:** no bits matched
- **State B:** 1 matched
- **State C:** 10 matched
- **State D:** 100 matched
- **State E:** 1001 matched
- **State F:** the full codeword 10010 was detected

Since 6 states were required, a 3-bit state vector was sufficient for this FSM. This allowed for a maximum of $2^3 = 8$ states.

The FSM begins in state A. On each clock cycle, the next state is determined by the current state and the incoming bit from the LFSR's LSB. As additional bits matching the codeword are received, the FSM advances through states B to F. If a mismatch occurs, the FSM switches to the state corresponding to the longest remaining valid prefix of the codeword.

State F is the only state for which the output is set to HIGH. This ensures that the output depends only on state, meaning it is a Moore FSM.

4.1: Initial Implementation:

The initial FSM implementation detected only non-overlapping occurrences of the codeword. In this version, the transition from state F on input 0 returned directly to state A. As a result, a sequence such as 10010010 would only be counted once, even though it contains two overlapping occurrences of the codeword.

Current State	Input X	Next State	Output Y
A	0	A	0
A	1	B	0
B	0	C	0
B	1	B	0
C	0	D	0
C	1	B	0
D	0	A	0
D	1	E	0
E	0	F	0
E	1	B	0
F	0	A	1
F	1	B	1

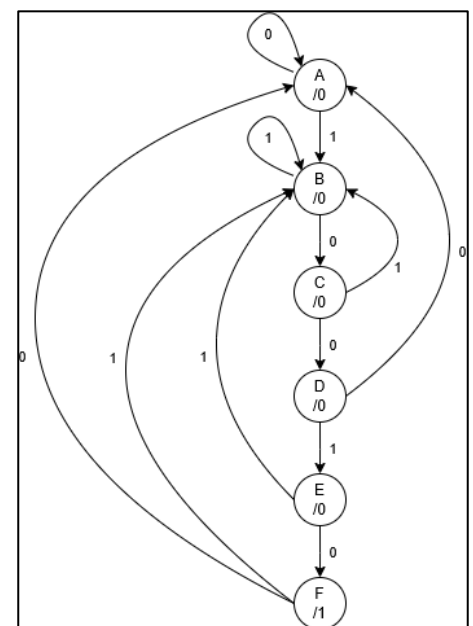


Figure 5 - Initial FSM Implementation

4.2: Corrected Implementation:

The FSM was then redone to support overlapping sequence detection. The corrected implementation changes the transition from state F on input 0 to state D rather than state A. This preserves the partial suffix 100, which is also a valid prefix of the codeword, allowing overlapping occurrences to be detected correctly. For example, the sequence 10010010 is now correctly counted as two detections.

Current State	Input X	Next State	Output Y
A	0	A	0
A	1	B	0
B	0	C	0
B	1	B	0
C	0	D	0
C	1	B	0
D	0	A	0
D	1	E	0
E	0	F	0
E	1	B	0
F	0	D	1
F	1	B	1

Table 1 - Corrected FSM Implementation

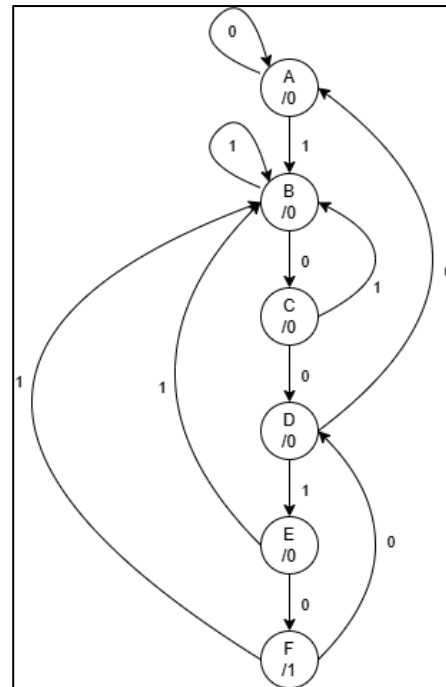


Figure 6 - Corrected FSM Implementation Diagram

4.3: State Minimisation:

State minimisation was then considered after deriving the FSM. For a Moore system, two states can only be merged if they produce the same output and transition to equivalent next states for all inputs. Although states A to E all produce an output of 0, each represents a different matched prefix of the target codeword and therefore has distinct transition behaviour. State F is also unique, as it is the only state with output 1. As a result, no states were equivalent and no further reduction in this system was possible, this is clearly shown in Table 1.

4.4: FSM Testing:

The FSM was tested using a custom testbench. The testbench included an initialising reset signal and an output signal for the current state, which was used for debugging. The following test cases were applied:

- **Initialisation:** Input held LOW during reset to ensure FSM starts in state A.
- **Non-Matching Sequence:** Random bits applied to ensure no incorrect detection.
- **Single Detection:** One codeword sequence applied to confirm output HIGH.
- **Reset during Detection:** Reset mid-sequence to ensure FSM returns to state A.
- **Back-to-Back Detection:** Two consecutive codeword sequences.

In these cases, the output was correctly set to HIGH only on detection of a full sequence. This expected behaviour is confirmed in the waveform in Figure 7.

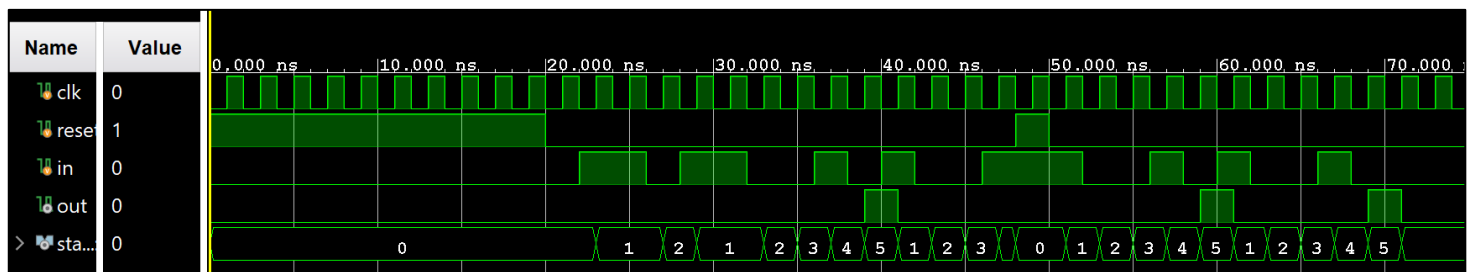


Figure 7 - FSM Testbench Waveform

5.0: Counter Design:

A binary counter was used to keep track of the number of codewords appearing in the LFSR output. As previously calculated, the LFSR design outputs 262,143 combinations, adding a new feedback bit each time. This means the total number of bits generated is:

$$262,143 \text{ bits}$$

In the worst case scenario, the LFSR repeatedly outputs the 5-bit codeword. In this case, the maximum number of codewords detected would be:

$$\frac{262,143}{5} \approx 52,428 \text{ codewords}$$

Therefore, the total length of the binary counter vector must have a length of:

$$\log_2(52,428) \approx 16 \text{ bits}$$

This scenario cannot actually occur in reality as the LFSR is always introducing new feedback bits and shifting the sequence, it could not repeatedly output the same 5-bit chain. However, it provides a simple upper-bound calculation for this purpose.

5.1: Counter Testbench:

This counter design was then verified using a dedicated testbench with the following test cases:

- **Initialisation:** Initial reset sets the counter to zero.
- **Incrementation:** Pulses applied to input increment the counter.
- **Hold Test:** When input is set to LOW, the counter value is maintained.
- **Reset Test:** Reset during operation returns counter value to zero.
- **Post-Reset Function:** Further pulses after reset continue incrementing counter.

The expected behaviour can be seen in Figure 8.

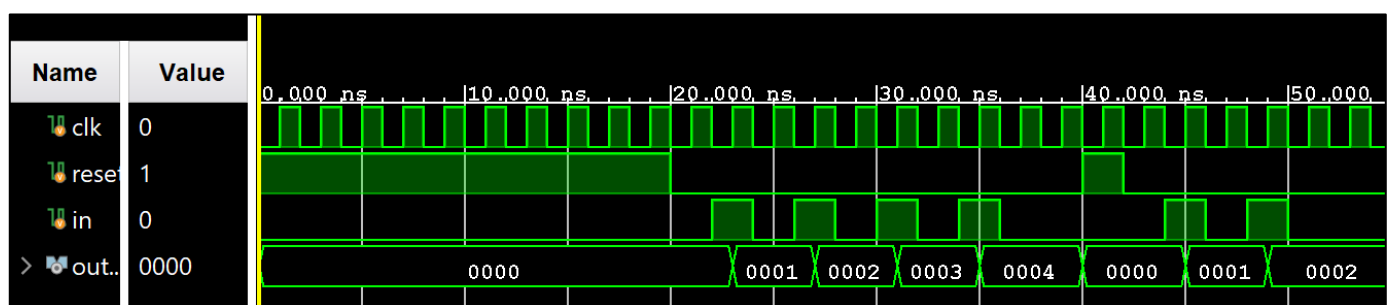


Figure 8 - Binary Counter Testbench Waveform

6.0: Implementation:

This system was implemented using a modular method, with separate modules for the LFSR, FSM, counter, clock divider, and output multiplexer. These modules were then integrated in a top-level module, as shown in the schematic in Figure 9.

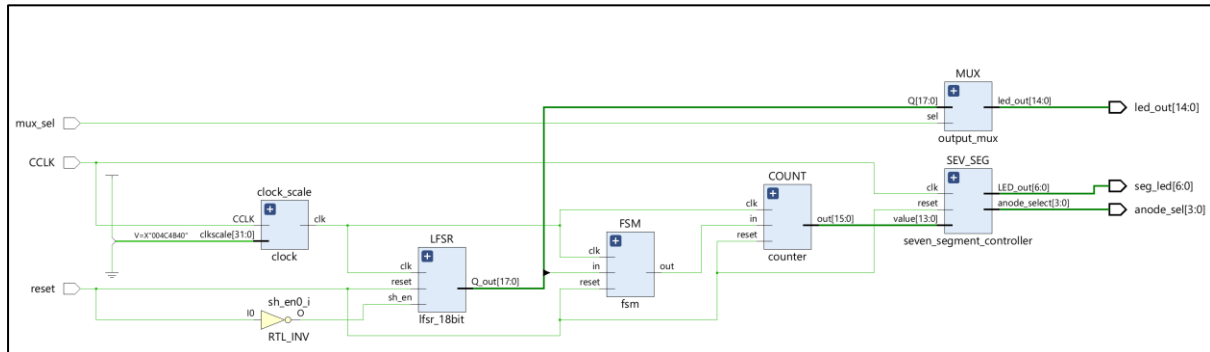


Figure 9 - Implemented Design Schematic

A synchronous, rising-edge design principle was used throughout the system. All updates, including those in the FSM, LFSR and counter, occur on the rising edge of the clock signal. This ensured consistent timing behaviour and avoided any race conditions. Combinational logic was used for state updating and output logic wherever appropriate.

6.1: Module Implementation & Hierarchy:

The top-level module instantiates the LFSR, FSM & counter modules, connecting them according to the system architecture (as shown in Section 2). The least significant bit of the LFSR output is used as the input to the FSM. The FSM output signal is then used to increment the binary counter.

Additional modules were included to support system operation:

- **Clock Divider:** used to generate a slower internal clock for the LFSR, FSM, and counter.
- **Multiplexer:** used to route either the first or second section of the LFSR output to the board LEDs for debugging.
- **7-Segment Display Controller:** used to display the counter value on the board.

The project hierarchy is shown in Figure 10, showing which sub-modules are instantiated in the top level module.

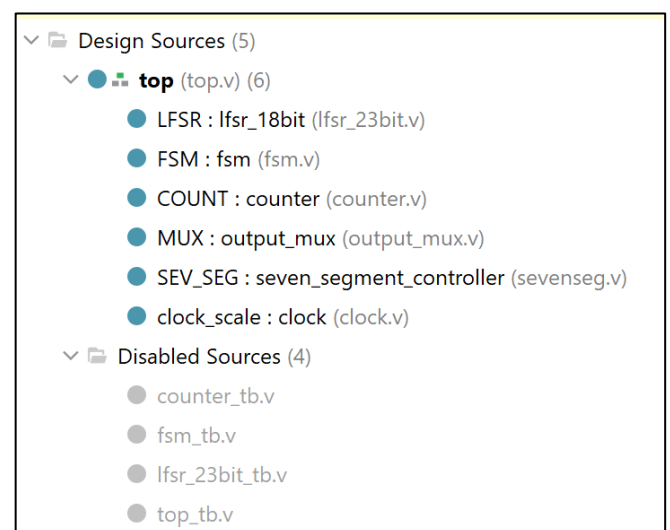


Figure 10 - Implemented Design Hierarchy

6.2: Design Decisions:

There were several design decisions made to improve output readability:

- **Clock Scaling:** Similar to “Lab F”, the LFSR and FSM operate on a reduced clock frequency so that the output behaviour is human-readable.
- **Reset Handling:** A global reset was used to initialise all of the modules to known initial states at the start of operation.
- **Debug Outputs:** The LFSR output was connected to LEDs using a multiplexer to allow real-time confirmation of the system behaviour on the Basys-3 Board.
- **7-Segment Display:** This module, given in a previous laboratory, was modified to use a 14-bit input, allowing all 4-digits to be used (representing values up to 9999).

All other design decisions were made in accordance with the assignment brief.

6.3: Implementation Testing:

As mentioned in Section 4, the LFSR cycle length is too long for practical real-world testing. The top-level module was tested using a simulation testbench because of this. This confirmed the correct interaction between modules, including sequence detection and counter incrementation.

Figure 11 shows the initialisation sequence of the implemented system. The LFSR, FSM and binary counter can all be seen functioning as expected. The counter is initialised at zero during the reset period and then begins incrementing at pseudo-random intervals when the codeword is detected.

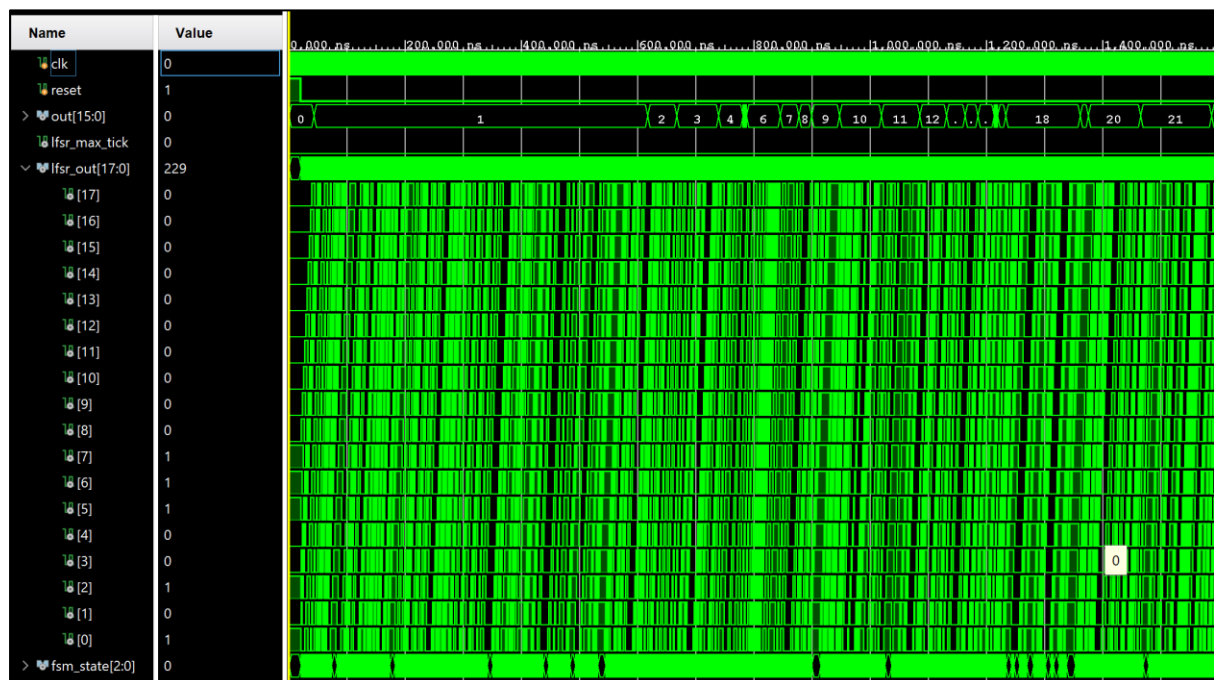


Figure 11 - Implementation Testbench – Initialisation

Figure 12 shows the final count after a full cycle of the LFSR. The “LFSR Max Tick” signal can be seen to be correctly set to high after ~524,300 ns. The final code-word count at this point is:

$$n = 8191$$

The expected value for this can be calculated by considering how many times a 5-bit code should appear among 262,143 different combinations:

$$n_{\text{expected}} = \frac{262,143}{2^5} = 8191.969$$

Therefore, the system seems to perform as expected, correctly detecting the codeword occurrences in a pseudo-random LFSR output.

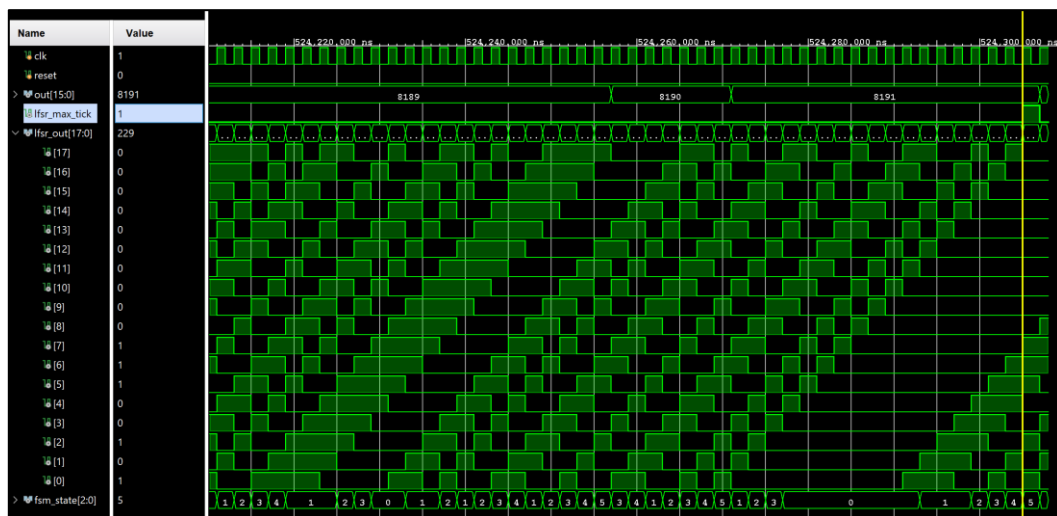


Figure 12 - Implementation Testbench - Final Count

The final test was to confirm the overlapping-codeword case as discussed in Section 4.2. This can be seen in Figure 13, where the overlapping case can be seen highlighted in red. It can be seen that the FSM correctly traverses to the final state (5) twice in only 8 bits. In the same sequence, the binary counter can be seen to increment twice, confirming the desired behaviour.

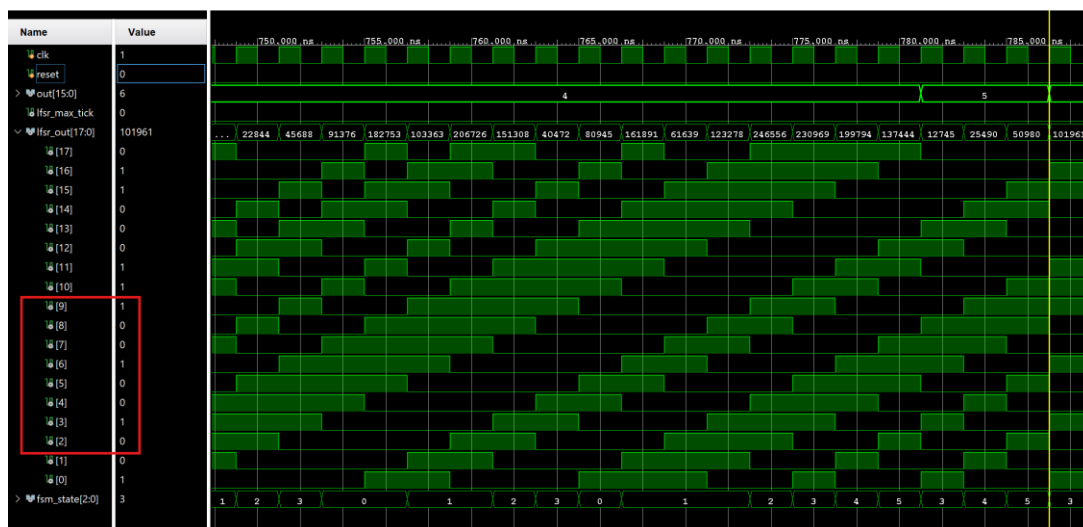


Figure 13 - Implementation Testbench - Overlapping Codewords

6.4: Clock Scaling:

The Basys-3 board clock was constrained in the XDC file as a 100 MHz input clock using “`create_clock -period 10.000`”. A clock divider module was then used to generate a slower internal clock for the LFSR, FSM, and counter. Since the divider toggles the output clock every “`clk scale`” input cycles, the output frequency is given by:

$$f_{out} = \frac{f_{in}}{2 \cdot \text{clk scale}}$$

Using `clk scale = 5,000,000` produced a 10 Hz internal clock. The 7-segment display was still connected to the Basys Board’s internal clock, as that module was designed to operate on a 100 MHz cycle. This is so each digit can be updated frequently enough so they all appear illuminated simultaneously.

The generated clock signal in the constraints file can be seen in Figure 14.

```
59 | ## Clock signal
60 | set_property PACKAGE_PIN W5 [get_ports CCLK]
61 | set_property IOSTANDARD LVCMOS33 [get_ports CCLK]
62 | create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports CCLK]
63 |
```

Figure 14 - Constraints File - Clock Signal Generation

7.0: Reporting & Testing:

7.1: Utilisation Report:

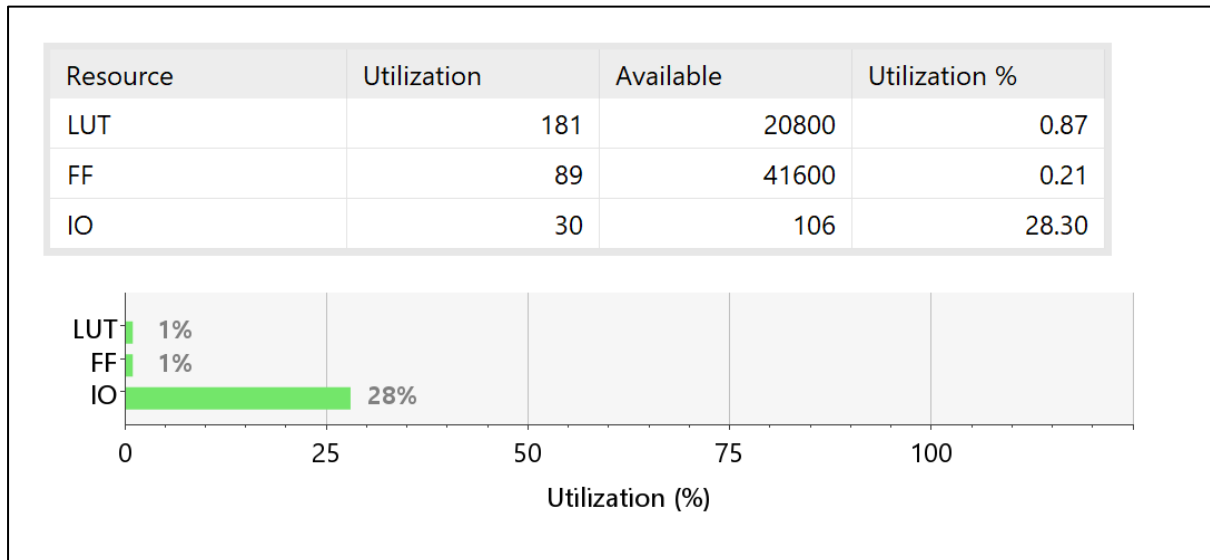


Figure 15 - Utilisation Report

The utilisation report can be seen in Figure 15. Very little logic is used with the LUT and flip-flop use at 0.87% and 0.21% respectively. This indicates that the design is not resource-intensive. The IO usage is considerably higher at 28% as the switches, every LED and the full 7-segment display are used in this design.

Overall, this design uses very little of the FPGA's available resources.

7.2: Timing Report:

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 3.862 ns	Worst Hold Slack (WHS): 0.263 ns	Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 85	Total Number of Endpoints: 85	Total Number of Endpoints: 54	
All user specified timing constraints are met.			

Figure 16 - Timing Report

The timing report can be seen in Figure 16. There are no failing endpoints reported in any category. The worst slack reports are all positive indicating that all signals are meeting the required clock period. This indicates that all timing constraints are met and the design was implemented successfully.

8.0: Demo:

Using the implemented design I/O pins were selected and a bitstream was then generated. The selected input and outputs can be seen pictured and labelled in Figure 17.

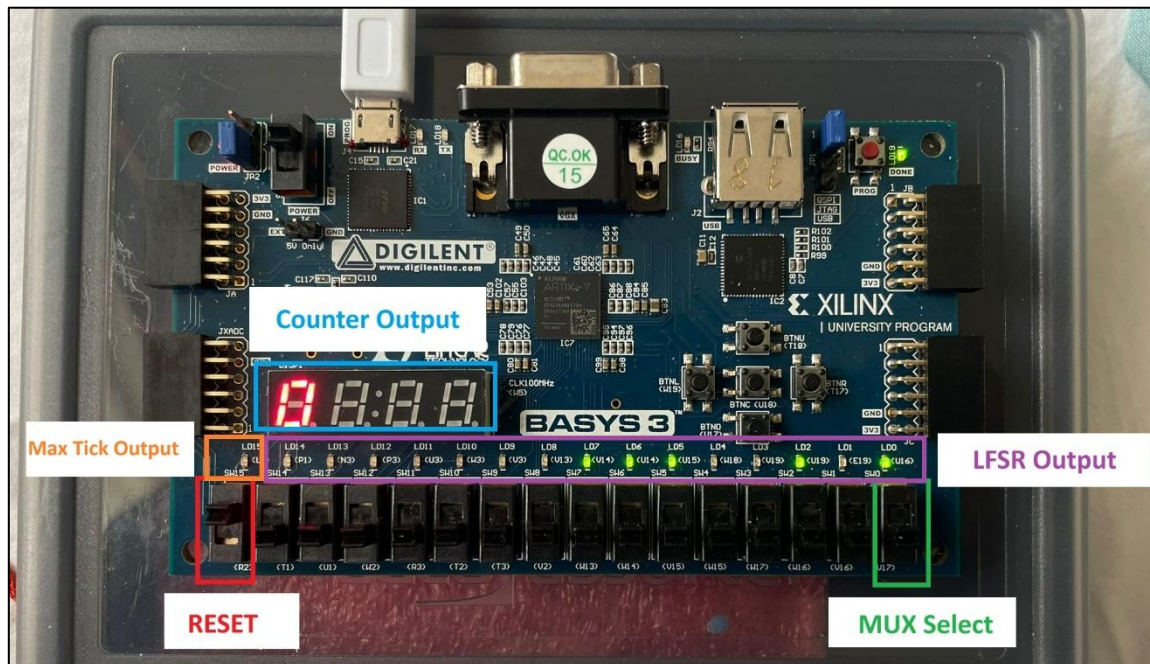


Figure 17 - Programmed Board with Labelled Inputs/Outputs

The 7-segment display is used to show the current counter value in human-readable format. It will show the current number of codeword occurrences in this LFSR cycle.

The LED strip is used to show the binary output of the LFSR. Except the left-most LED which is used as the maximum tick indicator. This will turn on for one clock cycle when the LFSR has cycled through all possible outputs and returns to its initial value.

The right-most switch controls the multiplexer select. Only 14 LEDs are being used to display the LFSR output, which is 18 bits long. The most significant 4 bits will be shown when the MUX select switch is set to HIGH.

The left-most switch controls the RESET input. When this is set to HIGH, the LFSR returns to its initial seed value and the counter is reset to zero (this can be seen in Figure 17).

The system can be seen running with a single codeword detection in Figure 18.

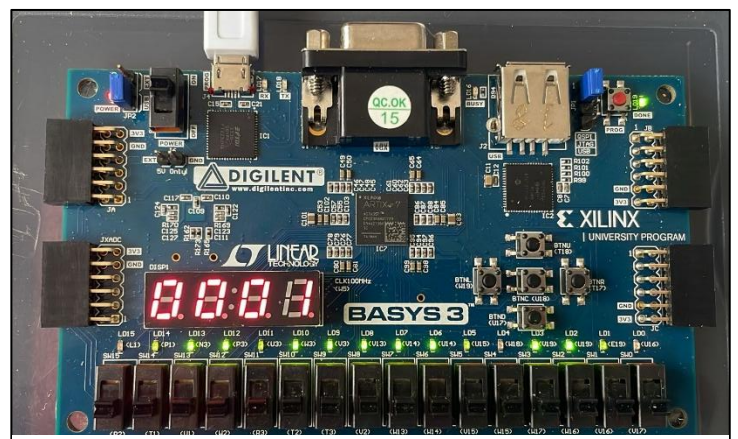


Figure 18 - System Demo – Single Codeword Occurrence

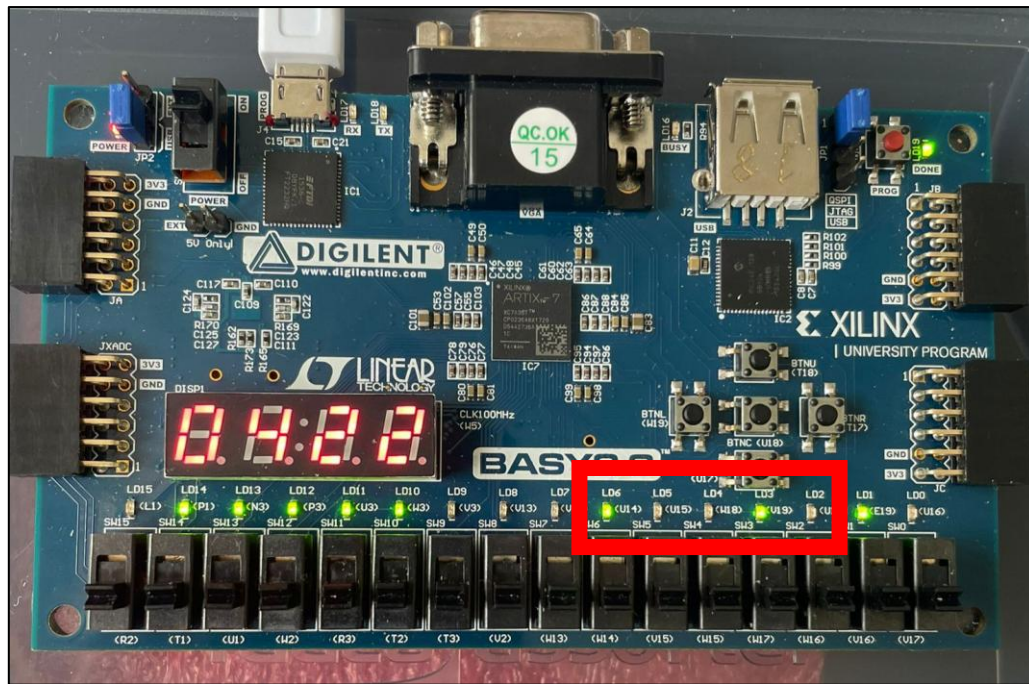


Figure 19 - System Demo - Codeword Highlighted

Another demo screenshot, with the codeword highlighted can be seen in Figure 19. This picture also demonstrates a much higher counter value.

Finally, the output multiplexer output multiplexer can be seen set to HIGH in Figure 20, showing the most significant four bits on the LED strip.

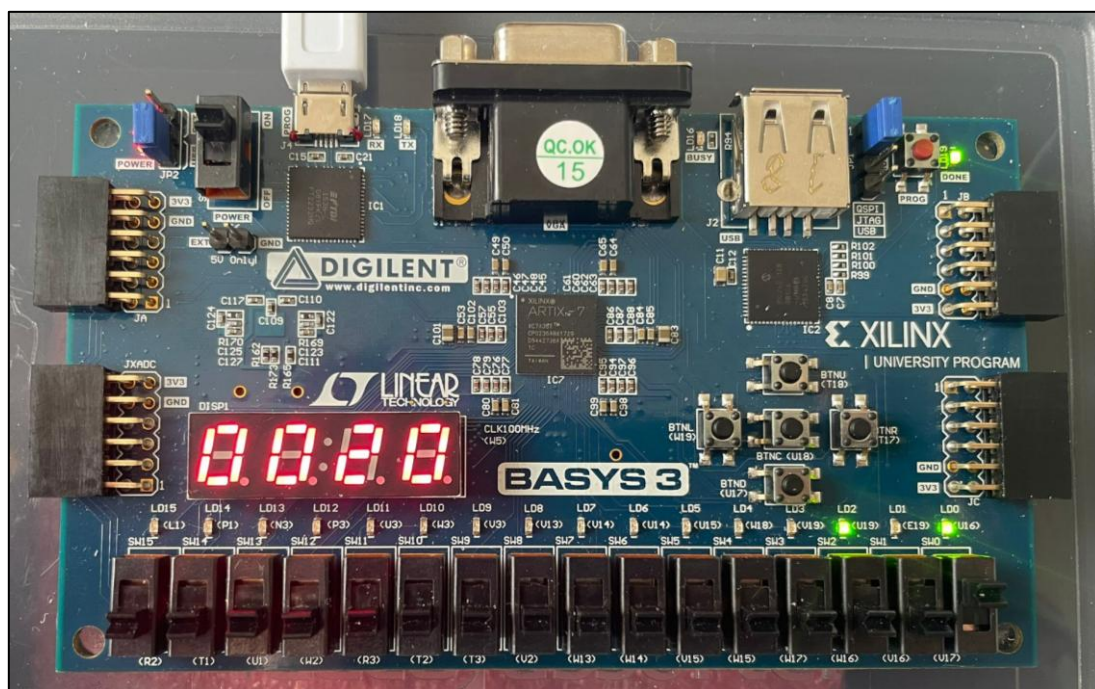


Figure 20 - System Demo - Output MUX